AegisBank 安全设计文档

仓库：https://github.com/0xChainAegis/aegisbank ｜ 配套：README、技术文档、安全自评报告、测试与复现

本文描述 AegisBank 的安全目标、威胁模型、信任边界与控制设计，与《安全自评报告》《测试与复现》共同构成安全交付材料。

所有金融数据来自确定性 Mock Bank Core，本原型不连接真实银行或支付网络。

1. 安全目标与设计原则

AegisBank 要让一笔金融操作从自然语言意图到工具执行全程可解释、可拦截、可追溯，具体保证五件事：

1. 意图不越权：模型只能生成候选计划，不能决定工具、权限、风险或执行；

2. 执行需确认：低风险自动执行，中高风险必须由用户明确确认，极高风险叠加二次验证；

3. 参数不漂移：用户确认的计划、参数、风险、权限与最终执行严格绑定；

4. 失败不伪装成功：未就绪、结果未知、补偿失败均 fail closed 或转人工核对；

5. 全程可追溯：指令、计划、参数、确认、执行结果与证据写入可校验的审计哈希链。

设计遵循以下原则：

原则	落地方式
模型不可信	模型只产出结构化候选计划；字段白名单、未注册工具拒绝、参数冲突校验；模型不参与风险与授权
默认拒绝	新工具、新用户默认无权限；未注册工具不可执行；未就绪时写操作返回 503
最小权限	每个工具声明最小权限；角色能力由服务端 AuthzService 统一判定
风险只升不降	最终风险取工具基线与确定性规则的最大值，模型输出不能降低风险
单一入口	所有工具与补偿统一经权限网关与 ToolExecutor，禁止直接调用 tool.execute
确认即绑定	中高风险动作生成 ActionEnvelope，执行前服务端复核计划与参数哈希、nonce、有效期
证据可验证	结果证据按来源分层写入，SHA-256 哈希链支持完整性校验

范围：六个参考场景（转账、账单、理财、卡片、订阅、跨场景联动）、51 个工具、Web 控制台与 IM / CLI / webhook 渠道；运行于本地 Linux / WSL 模拟环境。

2. 威胁模型

2.1 受保护资产

资产	说明
金融数据	账户、卡片、交易、账单、订阅、理财持仓与订单（均为 Mock Bank Core 模拟数据）
权限与角色	工具级授权、角色能力矩阵、撤权状态
待确认动作	计划快照、ActionEnvelope、nonce、状态机与执行 journal
审计记录	指令、DAG 路径、工具入参、执行结果、确认状态、证据与哈希链
执行环境	工具执行器、SandboxGuard、补偿工具、定时任务
模型与语义输入输出	用户原文、结构化计划、语义帧与澄清结果

2.2 威胁与攻击面

威胁	主要攻击面	主要控制
Prompt 注入 / 越权指令	自然语言指令、工具参数、IM 消息	规划前扫描指令与参数，命中即整条拒绝；节点级网关再检查；计入熔断
模型幻觉 / 编造对象与金额	LLM Planner 输出	金融事实只来自 Mock Bank Core；模型候选字段白名单、未注册工具拒绝、金额来源完整词元校验；歧义转澄清
未授权 / 越权调用	API、渠道适配器	单一权限网关；工具最小权限；AuthzService 角色能力；前端不硬编码授权
确认后参数被替换或重放	确认接口、待确认动作	ActionEnvelope 绑定计划 / 参数哈希、风险、权限、nonce、有效期；执行前复核；不一致 409
重复提交 / 重复扣款	POST /api/agent/run、确认接口、IM 消息	Idempotency-Key、动作结果回放、IM message_id 幂等、状态机
可疑行为滥用	连续失败、短时红色操作、注入尝试	按用户熔断；触发后写操作 fail closed；operator 解锁并写审计
工具副作用失控	工具执行	ToolExecutor + ExecutionGuard；超时、网络、入出体积、工具白名单
审计失真 / 删改	审计存储	SHA-256 哈希链；证据分层；GET /api/audit/verify 校验
服务未就绪被写入	启动 / 依赖故障	readiness 检查四组件；写操作 503；前端禁用写操作
补偿旁路	回滚 / 补偿	补偿工具同样经网关、风险复核与审计；不可逆动作阻止自动回滚
环境与凭据泄露	配置、仓库	不提交 .env、真实凭据与客户数据；一键启动使用独立模拟库

2.3 信任边界

[不可信] 用户输入 / IM 外部消息 / 模型输出 / X-Aegis-Role 请求头（演示身份）

|

▼

┌──┐

| 确定性控制面（可信） |

| 注入扫描 → 有界语义 / Planner → Plan Compiler → 动态风险 |

| → 统一权限网关 → ActionEnvelope / 确认 / MFA → DAG 调度 |

| → ToolExecutor + ExecutionGuard → 审计哈希链 |

└──┘

|

▼

[模拟] Mock Bank Core（确定性金融事实，SQLite）

信任边界内的不变量：

1. 模型输出只能是候选计划，不能包含工具、权限、风险、verified 或规范实体编号；

2. 风险等级只能保持或上调，模型不能降级；

3. 任何工具调用必须先经 PermissionGateway，再经 ToolExecutor；

4. 中高风险动作在执行前必须重新校验 ActionEnvelope；

5. 服务未就绪时写操作必须 fail closed；

6. 审计证据在操作链路中同步产生，不得事后补写。

3. 安全架构

接入层 Web 控制台 / IM / CLI / webhook

|

语义与规划 注入扫描 → 有界语义 → Planner（规则 / LLM / 自动回退）

| ExecutionPlan/v1（候选）

编译层 Plan Compiler：工具注册 / Schema / DAG / 环检测 / plan_hash

| CompiledPlan

决策层 动态风险引擎 → 统一权限网关（最小权限 / 角色 / 分级决策）

| allow / confirm / reauth / deny

信任层 ActionEnvelope / 幂等键 / 状态机 / readiness / 证据分层

|

执行层 DAG 调度 → ToolExecutor + ExecutionGuard（沙箱）

|

数据层 Mock Bank Core（SQLite）

|

审计层 SHA-256 审计哈希链 + 证据分层 + 完整性校验

- 接入层只做协议适配：IM / CLI / webhook 解析确认语法后调用同一 DagScheduler，不维护第二套权限判断；

- 决策与执行分离：权限网关只做决策，执行器只做执行，审计贯穿两者；

- 生成目录不作为源码：六个职责包由 scripts/assemble.py 组装到 .runtime/ 后运行。

4. 权限分级与授权设计

4.1 分级策略

级别	执行策略	典型操作
□ 绿色	最小权限满足后自动执行	余额、账单、持仓、订阅、联系人、生日等只读查询
□ 黄色	展示计划、金额、对象与风险原因，用户明确确认后执行	小额转账（当日累计含本次 ≤1000 元）、取消订阅、虚拟卡申请、还款日 / 限额调整、预算锁定、生日礼物预约
□ 红色	确认 + 演示 MFA（OTP 或模拟生物识别）	单笔 >5000 元或当日累计 >1000 元转账、卡片挂失 / 解挂、密码重置、理财申购、额度大幅调整

动态风险由确定性规则计算，输入事实包括：单笔金额、当日累计转账、产品风险等级、操作类型、申请单状态等；输出等级、评分与可解释原因。规则只能保持或上调工具基线风险。

4.2 工具与权限模型

- 每个工具继承 BaseTool，声明 risk_level、permission、输入 / 输出 Schema 与参数模型；有副作用的工具声明 compensable / compensation_tool；

- 当前工具目录 51 个（核心 10 + 场景 27 + 扩展 14），其中 15 个声明可补偿；

- 权限以工具级 capability 表达，例如 transfer:execute、card:security、billing:read；

- AuthzService 是能力校验的唯一口径，角色矩阵如下：

角色	能力	约束
user	tool:execute、permission:manage、audit:read、trust:inspect	唯一可发起银行工具执行的角色
operator	ops:manage、permission:manage、audit:read、trust:inspect	可解锁熔断、紧急撤权；不能执行银行工具

角色	能力	约束
auditor	audit:read、trust:inspect	只读审计与信任核查

撤权立即生效并写 role_revocation 审计；撤权后旧确认动作不能执行，后续计划在网关被阻断。

演示环境使用 X-Aegis-Role 请求头表达角色，服务端不验证身份；生产必须由认证后的会话 / Token 推导角色。

5. 确认、执行绑定与幂等

5.1 ActionEnvelope/v1

中高风险动作在执行前为每个计划节点生成 ActionEnvelope：

字段	含义
schema_version	固定 action-envelope/v1
user_id	确认者身份（演示期为固定演示用户）
plan_hash	编译后规范化计划的 SHA-256
step_id / tool_name	绑定的计划步骤与工具
arguments_hash	规范化参数的 SHA-256
risk_level / risk_reasons	确认时的风险等级与原因
permission	工具声明的最小权限
nonce	一次性随机串
issued_at / expires_at	签发与过期时间（默认 TTL 300 秒，可配置）
confirmation_state	pending / approved / rejected / expired
approval_token	演示审批令牌；API 与审计只返回掩码结果

执行前服务端复核：schema 版本、有效期、nonce 未使用、用户一致、计划哈希、步骤与工具、参数哈希、风险等级、权限声明；任一不一致返回 409，不执行、不创建业务流水、不写业务审计。

5.2 幂等与重复执行防护

- POST /api/agent/run 与确认接口接受 Idempotency-Key，按 scope 保存请求哈希与首次响应；

- 相同 key 相同请求直接回放首次结果，不产生第二个待确认动作或第二次执行；

- 相同 key 不同请求体返回 409；处理中返回 409，失败释放键允许重试；

- 确认动作持久化首次结果，重复确认同一 action_id 不会二次执行；

- IM 渠道以 channel + sender_id + message_id 哈希作为幂等键，重复投递回放首次回执。

5.3 状态机与未知结果

DRAFT → AWAITING_CONFIRMATION → AWAITING_REAUTH → APPROVED → EXECUTING → COMPLETED

| | | ├→ FAILED

├→ REJECTED ├→ REJECTED └→ EXPIRED └→ UNKNOWN

└→ EXPIRED └→ EXPIRED ├→ RECONCILED

└→ ROLLED_BACK

- 状态与迁移历史持久化，每次合法迁移写审计，非法迁移写拒绝审计并抛错；

- UNKNOWN 表示“已发出、结果不确定”，禁止自动重试，只能人工对账转 RECONCILED 或 ROLLED_BACK；

- 补偿失败或不可逆步骤被阻止时进入 UNKNOWN 并转人工处理。

5.4 Readiness 与 fail-closed

GET /api/health/ready 检查四项：sqlite（读写）、audit_chain（哈希链）、mock_bank_seed（模拟种子）、planner（初始化；非关键）。关键组件任一失败为 not_ready，写操作一律 503；前端在非 ready 时禁用任务提交、确认与时间快进。

6. 输入与模型安全

6.1 Prompt 注入防御

- 指令与参数在 Planner 之前扫描，命中越狱、忽略指令、绕过权限、系统提示词等模式即整条请求拒绝：不生成计划、不写业务状态，写 injection_blocked 审计，响应 status=blocked；

- 权限网关在节点级再次检查；

- 注入命中计入熔断规则；预览接口同样先拒绝注入请求；

- 这是字符规则基线，不替代专门的安全策略引擎或模型输入输出防护。

6.2 模型候选校验

planning/llm.py 把模型输出视为不可信候选：

1. 只接受 ExecutionPlan / PlanStep 契约字段，契约外字段或非 JSON 对象直接失败；

2. 模型返回未注册工具 → 失败，不进入执行；

3. 模型不得覆盖用户显式参数、不得丢弃高风险参数；

4. 模型不得补全用户未提供的收款方或金额；金额必须对应完整词元的确定性解析或经验证的规则计划；

5. 冲突转 needs_clarification，不接受“模型赢”；

6. auto 模式下模型失败回退规则 Planner，并显式标注。

6.3 语义安全与澄清

规则 / 模型 / 显式工具入口共用同一安全校验：可识别的条件、未支持组合、多目标、金额歧义与显式金额冲突返回澄清与空步骤，不创建待确认动作；不允许“截断后执行剩余动作”。该防线是有界规则，会保守拒绝部分合法表达，不构成任意自然语言的安全理解保证。

6.4 幻觉与输出契约

- 金融事实只来自 Mock Bank Core；联系人、账户、卡片、订阅、持仓、账单等对象必须来自当前用户的受控事实目录；

- 查询无记录时如实返回“无记录”，不编造；

- 工具输出强制 simulated=true、有限 JSON 值、工具名一致，并校验声明 Schema；不以空对象或缺失字段伪造金融事实；

- 金额、日期等规范化结果保留原文证据与来源，供审计复核。

7. 执行安全、沙箱与补偿

7.1 统一执行链

所有工具与补偿统一经 ToolExecutor：先按顺序执行 ExecutionGuard.before_execute，再调用工具，最后逆序执行 after_execute；违规先写审计再抛错。业务写入、输出契约校验、成功审计与步骤 journal 在同一事务内提交，故障整体回滚。

7.2 SandboxGuard（进程内演示级）

控制	默认值	行为
启用开关	开启	关闭时回退普通执行器
墙钟超时	5 秒	超时抛 SandboxViolation(timeout) 并写审计
入参上限	8 KB	超限拒绝并写 sandbox_blocked
出参上限	64 KB	超限拒绝结果
网络阻断	开启	按工具上下文限制 socket 调用
工具白名单	默认空	非空时只允许白名单工具

边界：这是进程内逻辑约束，不是 OS 容器；Python 线程无法强杀，超时后仍可能继续计算，但统一数据库事务会拒绝截止时间后的写入；文件、子进程与 C 扩展副作用需生产环境的容器 / 命名空间隔离。

7.3 回退、补偿与人工接管

- 工具声明 compensable / compensation_tool，当前登记 28 条补偿规格（15 可补偿、13 不可逆）；

- 回滚只处理 journal 中已执行且未补偿的节点，按相反顺序执行；

- 每个补偿执行前重新做动态风险、权限与 MFA 复检；被拒绝或需要二次校验时不自动执行，写 compensation_blocked 并转人工对账；

- 已执行步骤中存在不可逆操作时阻止整条自动回滚，计划标注“不可撤销”；

- 恢复设置前校验 journal、当前状态与同资源后续写操作，禁止旧快照覆盖新指令；

- DAG 支持节点间暂停 / 恢复、人工回退与执行 journal；确认、恢复、回滚、对账使用按动作的并发锁，冲突返回 409。

8. 审计与证据

审计记录包含：触发时间、原始指令、DAG 路径、工具名与入参、执行结果、用户确认状态、风险等级、证据以及 prev_hash / entry_hash。SHA-256 哈希链把相邻记录串联，GET /api/audit/verify 返回链完整性校验结果。

证据按来源与验证方式分层：

source: mock_bank | human_confirmation | policy_engine | audit_chain

verification_kind: tool_result | approval | policy_decision | hash_chain

verified: true | false

detail: 人类可读说明

- 绿色执行返回工具结果、网关决策与哈希链证据；确认执行额外包含人工批准证据；

- 失败证据标记 verified=false 并说明“未自动重试”；

- 证据参与哈希链计算，新增证据后链校验仍通过。

边界：哈希链用于检测日志改动，不等同数字签名、WORM 存储或独立第三方存证；拥有写库权限的人可能重算链。生产需写入只追加、可校验的合规存储，并引入外部审计锚点与签名。

9. 异常响应与运营安全

9.1 可疑行为熔断

规则	默认阈值	默认窗口	触发来源
连续失败	3 次失败	60 秒	工具执行失败
短时多次红色操作	3 次	300 秒	红色操作进入确认流程
注入命中累计	3 次	300 秒	injection_blocked

三类规则共用同一 open 状态与冷却时钟，按用户隔离；打开期间写操作统一返回 blocked，仅服务端明确声明为只读、绿色且使用 *:read 权限的纯查询可继续；operator 解锁并写 circuit_breaker_unlock 审计。

9.2 MFA 策略

Provider	类型	演示 token	说明
OTP	otp	固定 6 位演示码	默认 provider
模拟生物识别	biometric	固定演示 token	模拟人脸 / 设备确认，不连接真实身份服务

策略按风险等级与敏感工具给出允许 provider 与推荐默认值；执行时强制校验整条计划允许 provider 的交集，确认请求可指定 provider，未知 provider 返回 403；成功 / 失败均写审计。生产必须接入银行真实 MFA 并由服务端策略强制 provider 与会话身份绑定。

9.3 渠道安全

IM / CLI / webhook 是薄适配层：解析显式确认语法后调用同一 Planner、风险、网关、MFA 与审计链；适配器本身不做权限判断；消息幂等防止重复执行；未就绪返回 503。生产需校验平台签名并把外部身份映射到真实用户。

9.4 一键启动安全默认

start.sh / start.ps1 / start.cmd 默认只绑定本机，使用独立 Mock 数据库并保留已有数据；端口或启动锁冲突时明确拒绝，不复用、不终止未知服务；只有 ready 与审计检查通过才输出启动成功；停止时只清理本次启动的进程组。

10. 安全边界与生产化路线

10.1 本原型的已知边界

1. 固定演示用户，无真实登录、会话、设备绑定与租户隔离；

2. 固定演示 OTP 与模拟生物识别 token，不是生产 MFA；

3. X-Aegis-Role 是自报请求头，不构成身份认证；

4. 工具与数据均为 Mock Bank Core 模拟结果，不对接真实银行核心；

5. 沙箱是进程内演示级，不是 OS 容器 / 命名空间；

6. SQLite 单机存储，不满足高可用、WORM 与分布式并发要求；幂等键与 nonce 未实现 TTL 清理；

7. 注入检测是字符规则基线，语义能力有界，不承诺任意自然语言安全理解；

8. 未实现速率限制、CSRF、mTLS 与细粒度 ABAC；

9. IM 渠道未校验平台签名，外部身份未与真实用户绑定。

10.2 生产化清单

- 接入银行统一身份认证、真实 MFA 与设备指纹，替换固定演示用户、角色请求头与演示 token；

- 用策略引擎（OPA / Cedar）承载风险与权限策略，并落到真实身份与租户；

- 幂等键、nonce、渠道消息迁移到带 TTL 与分布式一致性的存储；

- 确认、证据与审计写入独立、只追加、可校验的合规存储，引入密钥管理与外部审计锚点；

- 工具执行迁移到容器 / 微虚拟机，配合 seccomp / AppArmor、网络策略与资源配额；

- 对输入输出做严格 Schema、金额与账户白名单校验，补充限流、告警与人工复核；

- 完成威胁建模、渗透测试、依赖供应链扫描与数据合规评审。

11. 安全验收与材料关系

控制域	自动验证入口	证据
权限分级 / 风险只升不降	后端 API、转账、自动化与 MFA 测试	《测试与复现》
注入整条拒绝 / 模型候选校验	信任层与语义安全测试	《安全自评报告》
Envelope / 幂等 / 状态机 / readiness	信任层、多轮转账与交付回归测试	《安全自评报告》
熔断 / 角色 / 撤权	熔断、权限与安全回归测试	《安全自评报告》
沙箱 / 补偿 / 暂停回退	沙箱与补偿测试、六场景脚本	《测试与复现》
审计链与证据分层	信任层测试、六场景脚本、GET /api/audit/verify	《安全自评报告》

当前验证基线：后端 671 项 pytest（36 个测试文件）、前端 168 项 Vitest（14 个测试文件）、组装器 28 项 unittest、六场景端到端 31/31 通过；工具目录 51 个、HTTP /api 路径 49 个。自动测试覆盖已知用例，不代表生产安全认证或任意自然语言均能安全执行；设计边界与本机验证命令见《安全自评报告》和《测试与复现》。